COSC 419/519B – Deep Learning
Winter 2025 T2
Dr. M. S. Shehata

A2 (20 points) – Due Feb. 9th, 2026

Focus: PyTorch, ANN, Optimization

1. [4 points] Setting Up PyTorch and Data
   1.1. **Importing PyTorch:** Import PyTorch and matplotlib. We will also use a manual seed of 42 to make our results reproducible
   1.2. **Create Synthetic Data:** We'll create a more challenging synthetic dataset that is not linearly separable. Generate 200 data points belonging to two classes using a 'moon' shape distribution, via `sklearn.datasets.make_moons`.
   1.3. **Visualize the Data:** Use Matplotlib to create a scatter plot that visualizes the two classes in the dataset. Use the color 'blue' for Class 0, and use the color 'orange' for Class 1. Label the axes (e.g., "Feature 1" and "Feature 2"). Add a title to the plot, "Visualization of the moons Dataset."
   1.4. **Splitting the Dataset:** Split the dataset into 80% training and 20% test using slicing (not a train_test split).
   Convert the input features (X) to a torch.FloatTensor and the labels (y) to a torch.LongTensor.
   Print the shapes of the the training and testing data splits for verification.

2. [2 points] Building a Neural Network
   2.1. **Define a Simple Network Clas:** Create a class SimpleNN that inherits from nn.Module in PyTorch. This class should define a neural network with the following layers:
      - A linear layer that takes the 2 features of our data and produces 8 outputs.
      - A ReLU (Rectified Linear Unit) activation function.
      - A linear layer that takes 8 inputs and produces 2 outputs (for our 2 classes).

      Use the nn.Linear and nn.ReLU classes from PyTorch.

      Instantiate an object of this class.

   2.2. **Forward Pass:** Pass a single data point from the training set through the model and print the resulting output. What do these outputs represent?

3. [3 points] Training the Neural Network
   3.1. **Define Loss Function and Optimizer:**
      - Define the loss function as nn.CrossEntropyLoss.
      - Define the optimizer to use the optim.Adam optimizer with a learning rate of 0.01.
   3.2. **Training Loop**: Implement a training loop
      - Set the number of epochs to 200.
      - For each epoch:
         - Set the gradients to zero with optimizer.zero_grad().
         - Perform the forward pass with the training data X_train.
         - Compute the loss using the model predictions and the labels y_train.
         - Perform the backward pass (loss.backward()) to compute the gradients.
         - Update the parameters with optimizer.step().
         - Print the loss every 20 epochs.

      Store training losses in an array called losses, and after training plot the loss.

3.3. **Evaluation:** Compute the accuracy of the trained model on the test set X_test, y_test.
- Use torch.argmax() to obtain the predicted classes from the model outputs.
- Compute the number of correct predictions.
- Calculate accuracy.

4. [3 points] Deep Dive into PyTorch Optimizers

4.1. **Questions:** Answer the following questions**:**

a) What is the purpose of the params argument in the __init__ method? How does the optimizer use it to keep track of the model's parameters?

b) Explain the structure of param_groups. What is a parameter group, and why is it a list of dictionaries?

c) What kind of information is stored in the state attribute? How does the optimizer use it during the optimization process?

d) Why is it essential to call zero_grad() before computing gradients in each iteration of the training loop? What would happen if you didn't?

e) Briefly describe the role of the step() method. What does it do in an abstract sense (without going into the details of specific optimization algorithms)?

4.2. **Implementing a Basic Optimizer: DummyOptimizer**

Objective: Implement a simplified optimizer called DummyOptimizer to solidify your understanding of the Optimizer class's core methods.

- Create a basic optimizer class called DummyOptimizer that inherits from torch.optim.Optimizer.
- Implement the __init__ method:
  - Take params and a learning rate lr as arguments.
  - Store them in defaults.
  - Call the super() constructor to properly initialize the base class.
- Implement the step method:
  - Iterate through param_groups.
  - Iterate through the parameters (p) in each group.
  - If a parameter has a gradient, simply update the parameter by subtracting the learning rate times the gradient: p.data.add_(-group['lr'], p.grad.data)
- This DummyOptimizer essentially performs a very simplified version of Stochastic Gradient Descent (SGD).

4.3 **Testing the DummyOptimizer:** Test your DummyOptimizer implementation to ensure it's working correctly, by following the below steps:
- Instantiate your SimpleNN model.
- Instantiate your DummyOptimizer, passing the model's parameters and a learning rate.
- Modify the training loop from Task 3.2 (Training Loop) to use your DummyOptimizer.
- Run the training loop for 50 few epochs.
- Print the loss every few epochs to verify that the model is learning

5. [10 points] Implementing AdamW with Decoupled Weight Decay

5.1. **Answer the below questions**

a) Explain the difference between weight decay and L2 regularization. Why are they not equivalent in optimizers like Adam?

b) How does AdamW decouple weight decay from the gradient update? Explain the key difference in the update rule compared to the original Adam.

c) Based on the paper, why does this decoupling of weight decay tend to improve the model's generalization performance?

5.2. **Implementing AdamW**

Objective: Implement the AdamW optimizer with decoupled weight decay based on the paper and integrate a learning rate scheduler.

1. Create a new class called AdamW that inherits from torch.optim.Optimizer.
2. Implement the __init__ method:
3. Initialize parameters: learning rate (lr), betas (betas), epsilon (eps), and weight decay (weight_decay).
4. Store these in defaults.
5. Initialize state variables (exp_avg, exp_avg_sq) as in Adam.
6. Implement the step method:
7. Follow the AdamW algorithm from the paper (and the provided pseudo-code).
8. The key difference from Adam is the decoupled weight decay step: p.data.mul_(1 - group['lr'] * group['weight_decay'])
9. Use @torch.no_grad() to prevent gradient tracking during parameter updates.

5.3. **Integrating a Learning Rate Scheduler**

Objective: Integrate a learning rate scheduler into the training loop when using your custom AdamW optimizer.

- Modify the training loop to include a learning rate scheduler.
- Use torch.optim.lr_scheduler.StepLR to implement a step decay schedule.
- Reduce the learning rate by a factor (e.g., 0.1) every specific number of epochs (e.g., 50).

Testing and Comparing Optimizers

Objective: Compare the performance of your custom AdamW optimizer with the standard torch.optim.Adam and torch.optim.AdamW optimizers.

Instructions:

1. Train your SimpleNN model using your custom AdamW optimizer with the integrated learning rate scheduler.
2. Train the model again using the standard torch.optim.Adam optimizer (with a learning rate scheduler).
3. Train the model once more using the standard torch.optim.AdamW optimizer (with a learning rate scheduler).

4. Keep the initial learning rate the same across all optimizers (or use the same range in a grid search) for a fair comparison.
5. Plot the training loss curves for all three optimizers on the same graph.
6. Compute and print the final training loss and test accuracy for each optimizer.
7. Experiment with different values of weight_decay in your AdamW implementation (e.g., 0, 0.01, 0.1) and observe the effects on the model's performance.
8. Analyze the results:
   (a) How does the performance of your AdamW compare to Adam and torch.optim.AdamW?
   (b) What is the effect of different weight_decay values on the model's training and generalization?

Submission Instructions

1- Each question is linked to a cell in the provided notebook, provide your code whenever you see a comment "Your code here". (no group work).
2- Create text cells to answer the theoretical questions whenever necessary.
3- Submit the notebook to Canvas. Make sure your whole notebook runs without errors. Note that you can resubmit an assignment, but the new submission overwrites the old submission and receives a new timestamp.